

CS4432: Database Systems II

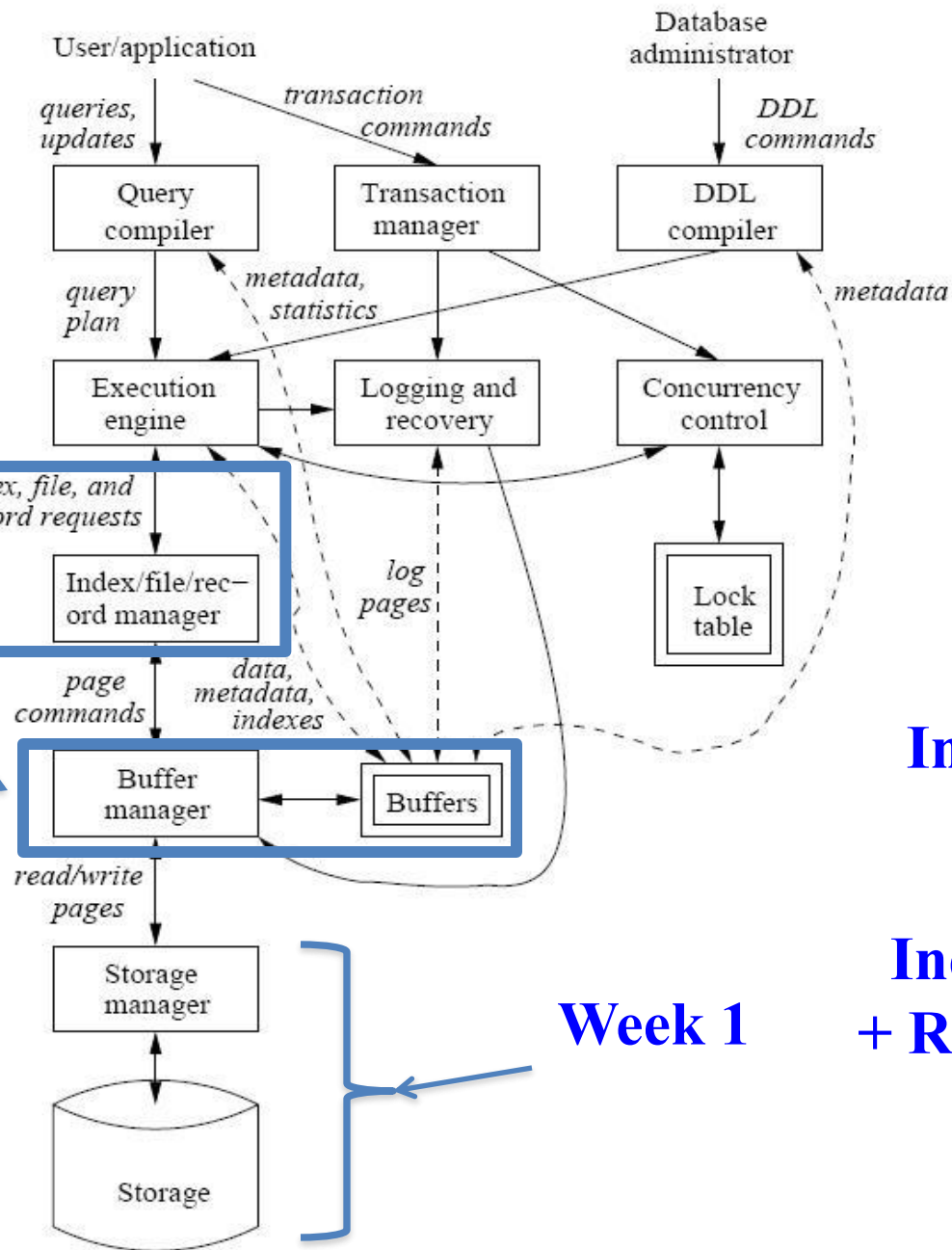
Query Processing- Part 1

Updates

- **The final Exam is next week (April 31st).**
 - **START STUDYING.**
- **HW 3 and Project 3 are out and due May 5.**
 - **Make sure you submit them on time**

Week 3:
sorting +

Week 2



Week 4:
Index via B+Tree

Week 5:
**Index via Hashing
+ Relational Algebra**

Core Relational Algebra (Recap)

- **Union, intersection, and difference.** $R \cup S$ $R \cap S$
 - Usual set operations, both operands have the same relation schema.
- **Selection:** picking certain rows. $\sigma_c(R)$
- **Projection:** picking certain columns. $\Pi_{B,D,\dots}(R)$
- **Products** and **joins:** compositions of relations. $R \times S$ $R \bowtie_c S$
- **Grouping** and **Aggregation:** Grouping matching tuples $\gamma_{g1,g2,\dots,gm, F1(A1), \dots, Fn(An)}(R)$
- **Duplicate Elimination:** eliminates identical copies except one $\delta(R)$
- **Sorting:** Orders tuples based on a given criteria $\tau_x(R)$

Week 6

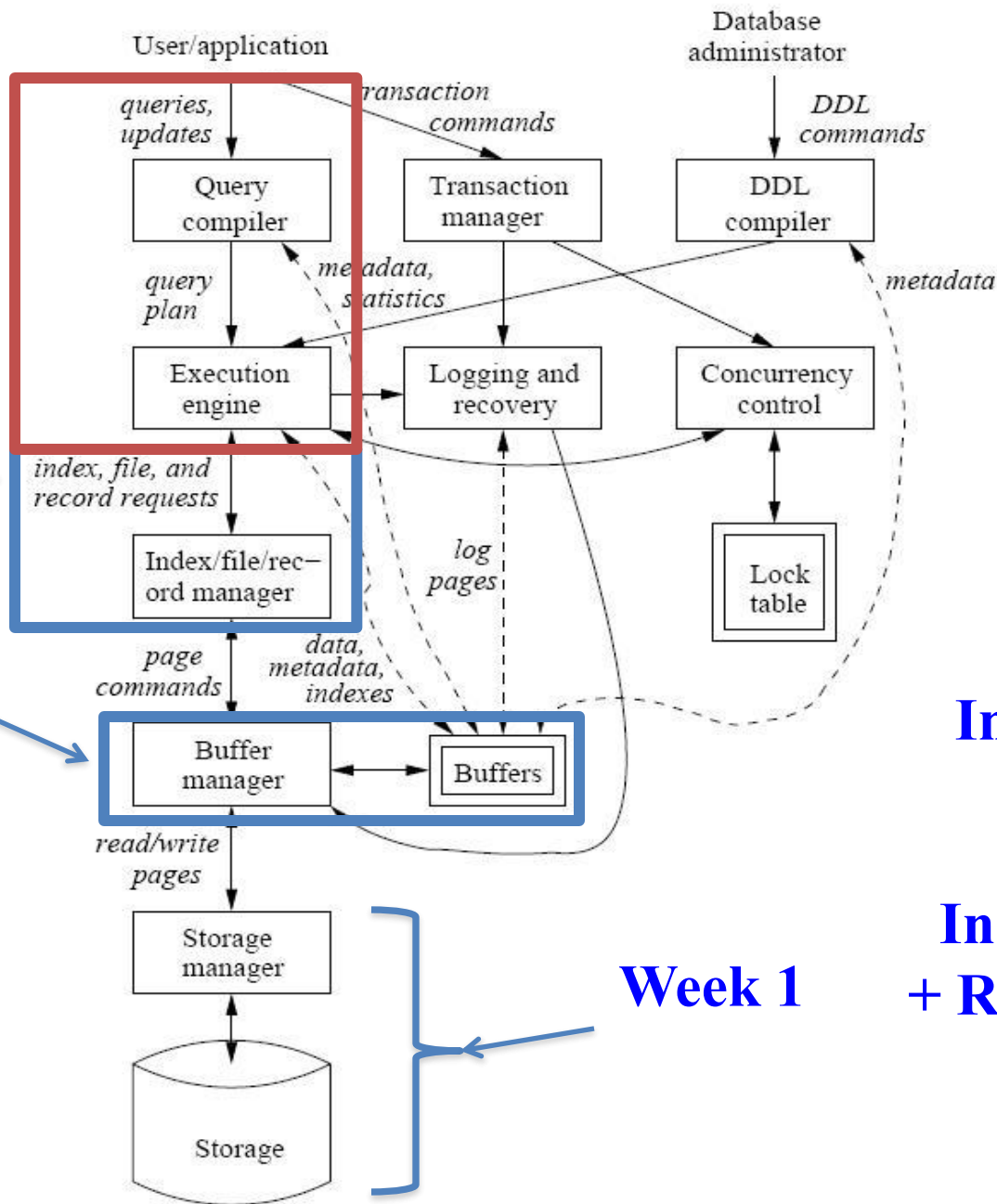
**Week 3:
sorting +**

Week 2

Week 1

**Week 4:
Index via B+Tree**

**Week 5:
Index via Hashing
+ Relational Algebra**



Database management system components

Learning objectives

- Be able to list different query plans for the same query
- Recognize if two query plans correspond to the same query
- Execute small query plans manually
- Identify concepts used when evaluating operators:
 - indexing, iterating and partitioning
 - Number of passes (e.g., 1, 2, multiple)
 - Blocking vs. non-blocking
- Identify important stats (& notation) for query planning
- Reason about the assumptions and I/O cost of operators (especially joins) under different scenarios

Example

Data:

relation R (A, B, C)

relation S (C, D, E)

Query:

SELECT B, D

FROM R, S

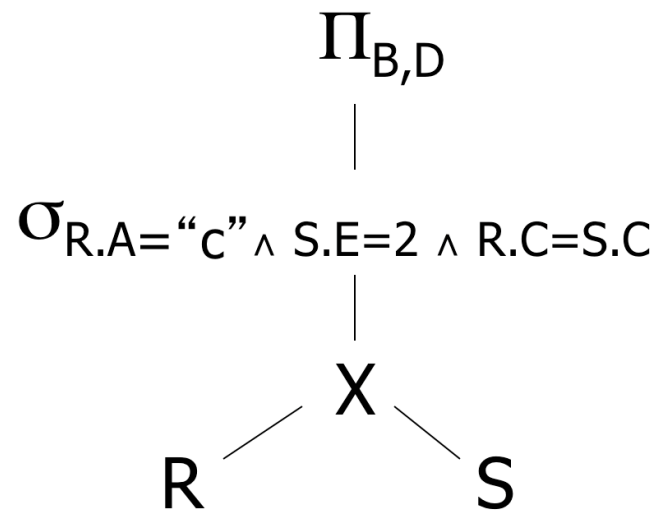
WHERE R.A = "c" and S.E = 2 and R.C=S.C

OR: $\Pi_{B,D} [\sigma_{R.A="c" \wedge S.E=2 \wedge R.C=S.C} (R \times S)]$

Relational Algebra – Possible Query Plans

Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C = S.C

Plan 1

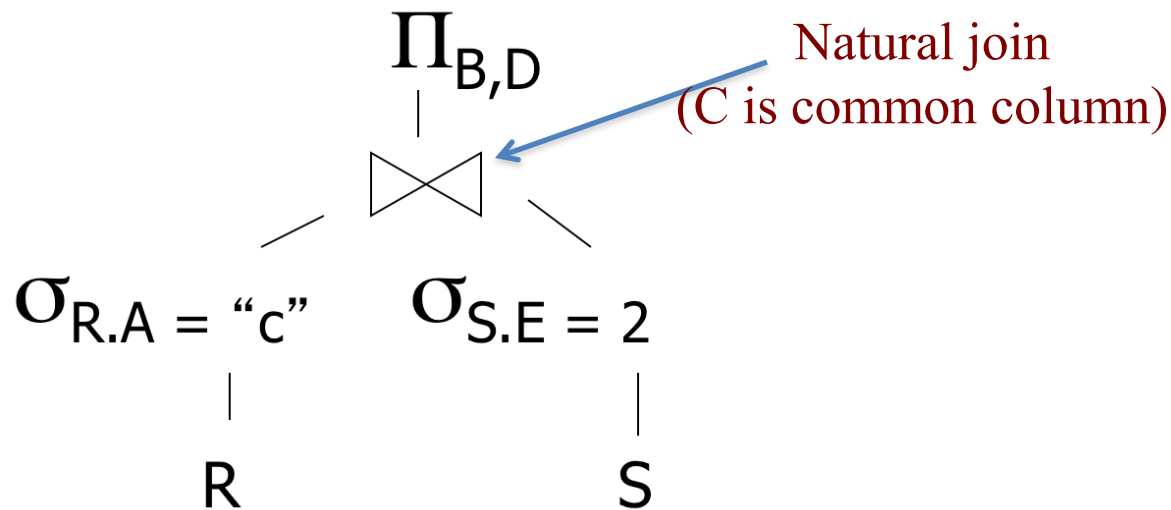


OR: $\Pi_{B,D} [\sigma_{R.A="c" \wedge S.E=2 \wedge R.C=S.C} (R \bowtie S)]$

Relational Algebra – Possible Query Plans

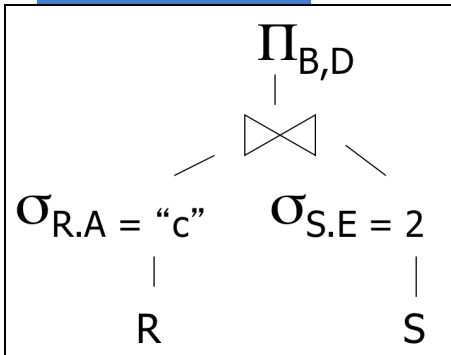
Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C=S.C

Plan 2



Plan 2

Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C=S.C



R

A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45

$\sigma(R)$

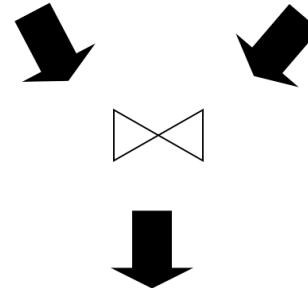
A	B	C
c	2	10

$\sigma(S)$

C	D	E
10	x	2
20	y	2
30	z	2

S

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3

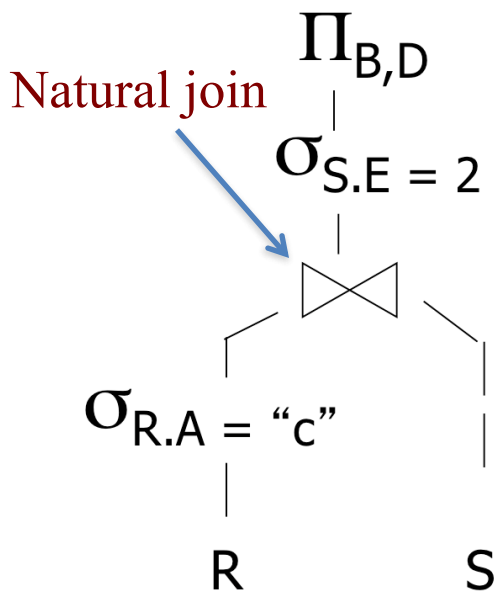


Relational Algebra – Possible Query Plans

Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C=S.C

Plan 3

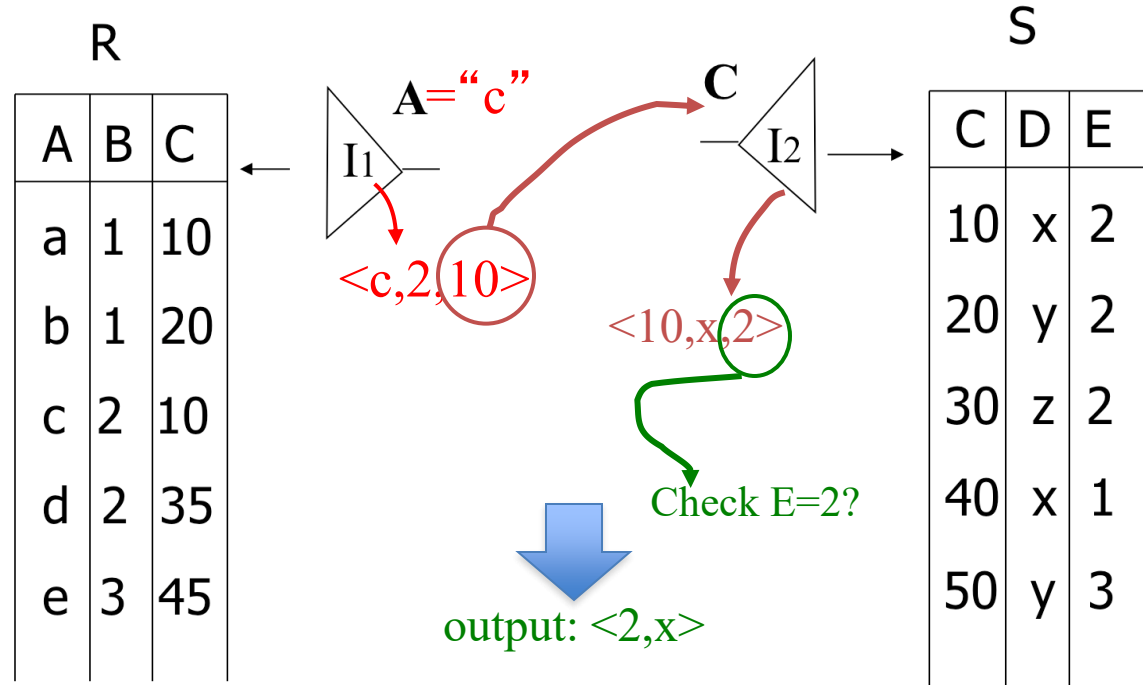
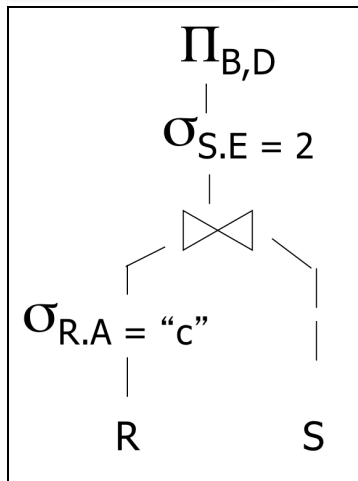
Assume indexes on R.A and S.C



- (1) Use R.A index to select R tuples with R.A = "c"
- (2) For each R.C value found, use S.C index to find matching tuples
- (3) Eliminate S tuples S.E $\neq$ 2
- (4) Project B,D attributes and place in result

Select B, D **From** R, S **Where** R.A = "c" **And** S.E = 2 **And** R.C=S.C

Plan 3



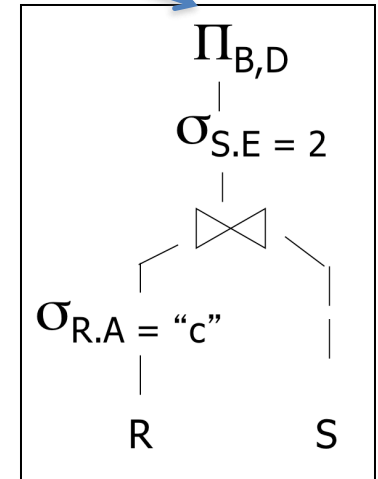
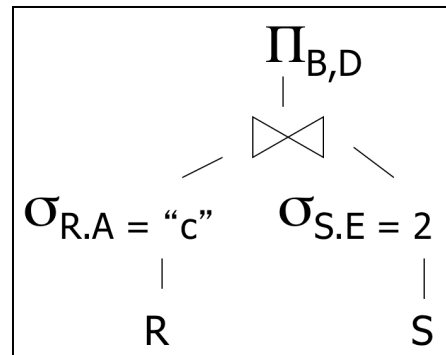
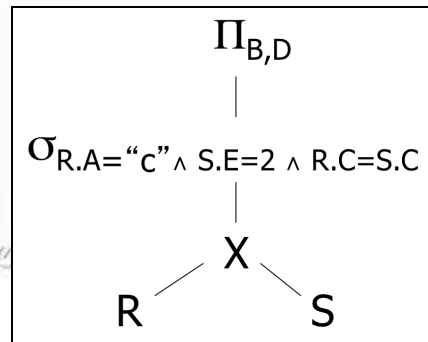


Select B, D

From R, S

Where R.A = "c"

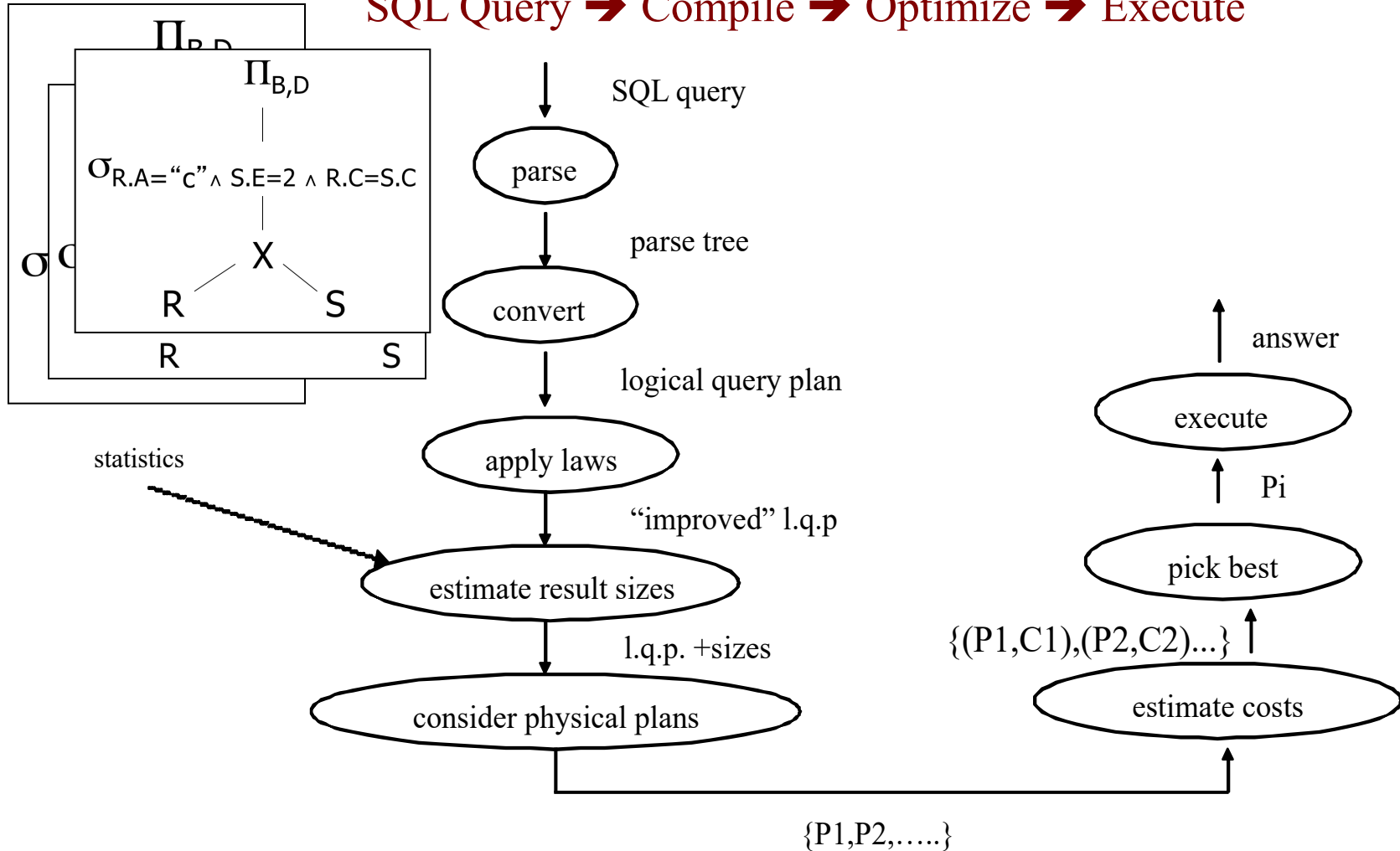
And S.E = 2 **And** R.C = S.C



How to reach from top to bottom?
How to select the best one?

Overview of Query Execution

SQL Query $\rightarrow$ Compile $\rightarrow$ Optimize $\rightarrow$ Execute



Improving Logical Query Plans



Introduction

- How to apply the algebraic laws to improve a logical query plan?
- Goal: minimize the size (number of tuples, number of attributes) of intermediate results.
- Push selections down in the expression tree as far as possible.
- Push down projections, or add new projections where applicable.

Improving Logical Query Plans



Pushing Selections

- Replace the left side of one of these (and similar) rules by the right side:

$$\sigma_{p1 \wedge p2} (R) \rightarrow \sigma_{p1} [\sigma_{p2} (R)]$$

$$\sigma_p (R \bowtie S) \rightarrow [\sigma_p (R)] \bowtie S$$

- Can greatly reduce the number of tuples of intermediate results.

Operator Evaluation

- Two categorizations:
 - Type of Technique/Algorithm
 - Number of Passes

Techniques For Evaluating Operators

Algorithms for evaluating *relational operators* use some simple ideas extensively:

- **Indexing**
- **Iteration**
- **Partitioning**

Techniques For Evaluating Operators

Algorithms for evaluating *relational operators* use some simple ideas extensively:

- **Indexing:** Can use WHERE conditions to retrieve small set of tuples (selections, joins)
- **Iteration:** Sometimes, faster to scan all tuples even if there is an index. (Sometimes we can scan the data entries in an index instead of the table itself (when?).)
- **Partitioning:** By using sorting or hashing, we can partition the input tuples and replace an expensive operation by similar operations on smaller inputs.

Another Categorization

- **One Pass Algorithms**

- Need one pass over the input relation(s)
- Puts limitations on the size of the inputs vs. memory

- **Two Pass Algorithms**

- Need two pass over the input relation(s)
- Puts limitations on the size of the inputs vs. memory

- **Multi-Pass Algorithms**

- Scale to any size and may need several passes over the input relation(s)



Common Statistics over Relation **R**

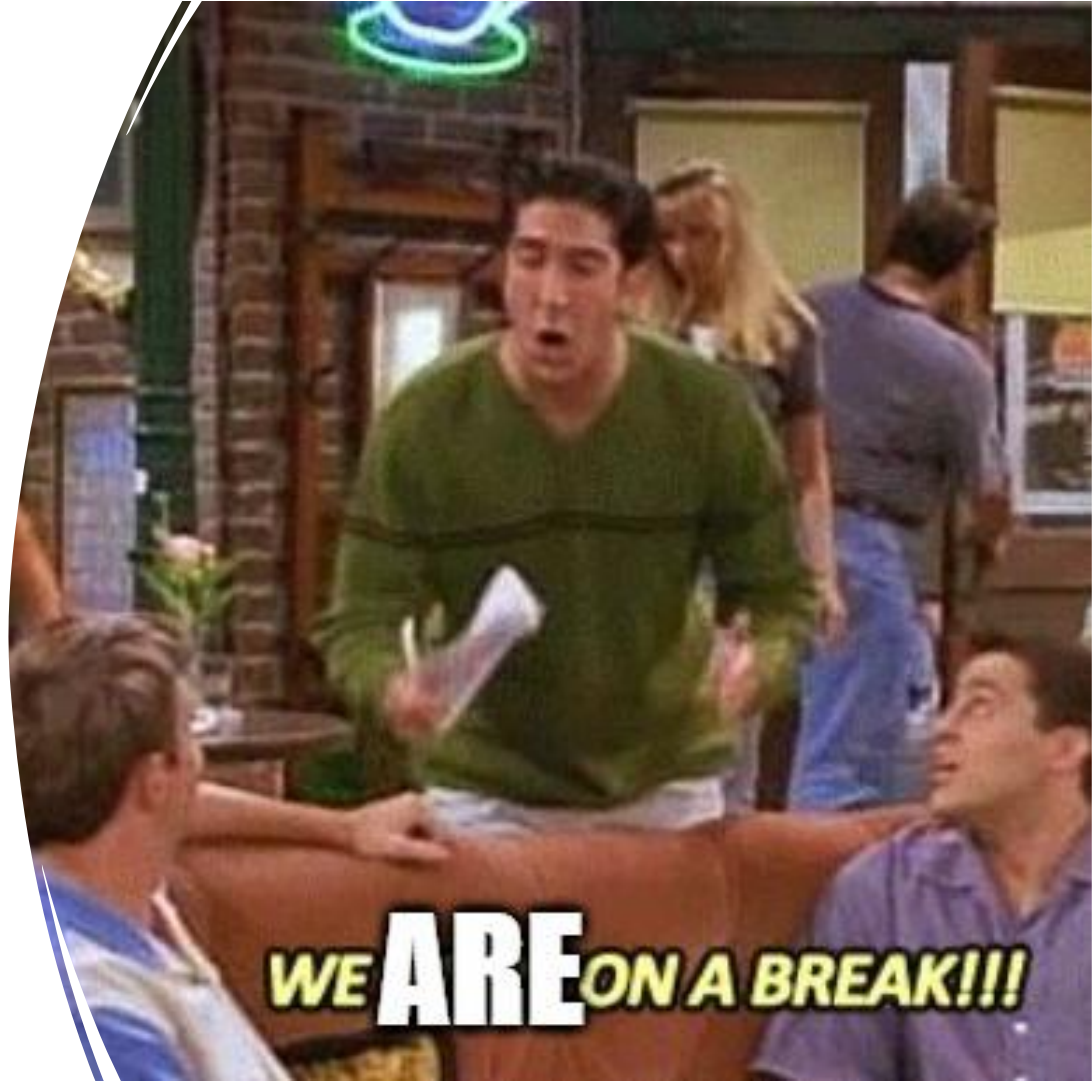
- **M**: # of memory buffers available
- **B(R)**: # of blocks to hold all R tuples

In the next slide decks, also:

- **T(R)**: # tuples in R
- **S(R)**: # of bytes in each of R's tuple
- **V(R, A)**: # distinct values in attribute R.A

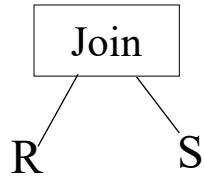
Break

- Now we will take a few minutes break



Rest of class

- Operator Evaluations that are:
 - One Pass
 - Two Pass



Example: Join (R,S)

One Pass

Iteration

Assume S is smaller than R
i.e., $B(S) < B(R)$

Open():

read S into memory

GetNext():

```
for b in blocks of R:
  for t in tuples of b:
    if t matches tuple s:
      return join (t,s)
return NotFound
```

Close():

Clean memory

- For this join algorithm to work:
 - S must fit in memory
 - One additional buffer for R

- **Key Metrics (memory Req.):**
 - $M \geq B(S) + 1$
- **I/O Cost:**
 - $B(S) + B(R)$

Distinct

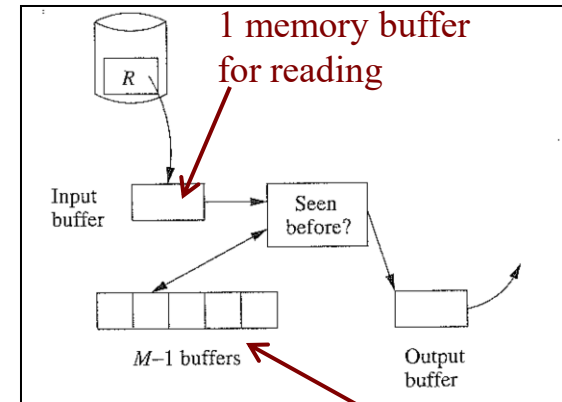
R

Example: Duplicate Elimination

One Pass

Iteration

- Keep a main memory search data structure D (use search tree or hash table) to store one copy of each tuple → **(M-1 Buffers)**
- Read in each block of R one at a time (use table scan) → **(1 buffer)**
- For each tuple check if it appears in D
 - If Yes, then skip
 - If Not, then add it to D and to the output buffer



What are the constraints for this algorithm to work in one pass?

The distinct tuples of R must fit in $M-1$ Buffers

>> $B(\text{Distinct}(R)) \leq M-1$

>> As an approximation $B(\text{Distinct}(R)) \leq M$



What is the I/O Cost

$B(R)$

Distinct

R

Example: Duplicate Elimination

- **What if relation R is sorted**
- **How the duplicate elimination op. works ?**
- **Are there any size constraints to be in one pass ?**
- **What is the I/O cost ?**

Distinct

R

Example: Duplicate Elimination (Cont'd)

- **What if relation R is sorted**
 - **How the duplicate elimination op. works ?**
 - No need for the M-1 Buffers (we keep only the last reported tuple)
 - **Are there any size constraints to be in one pass ?**
 - No (1 memory buffer to handle R of any size)
 - **What is the I/O cost ?**
 - $B(R)$
- Each operator must know the properties of its input relations (Sorted or not, grouped or not, ...)
 - Makes big difference in execution and performance

Group By

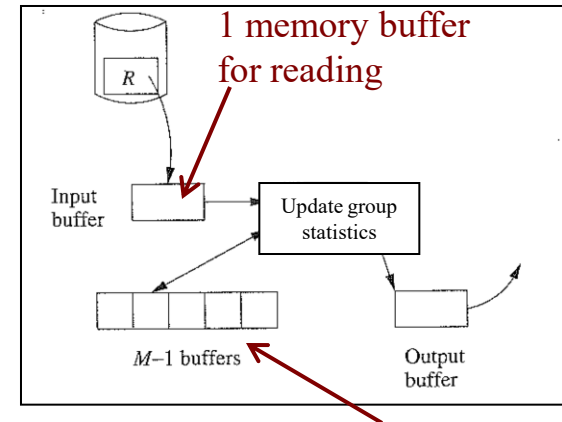
R

Example: Group By

One Pass

Iteration

- Keep a main memory search data structure D (use search tree or hash table) to store one entry for each group $\rightarrow$ (**M-1 Buffers**)
- Read in each block of R one at a time (use table scan) $\rightarrow$ (**1 buffer**)
- For each tuple, update its group statistics



M-1 memory buffers for storing one entry for each group



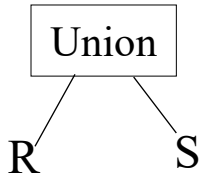
What are the constraints for this algorithm to work in one pass?

- The groups must fit in M-1 buffers
- $B(\text{Group}(R)) \leq M-1$
- **Worst case:** Each tuple is a group



What is the I/O Cost

$B(R)$



Example: *Set* Union(R,S)

Implicitly eliminates duplicates

One Pass

Iteration

Assume S is smaller than R

- Read smaller relation into main memory (S) → **M-1 Buffers**
- Use main memory search structure *D* to allow tuples to be inserted and found quickly
- Produce S's tuples to output as you read them
- Read from R one block at a time → **1 Buffer**
 - If tuple exists in *D*, skip
 - Otherwise, write to output



What are the constraints for this algorithm to work in one pass?

$\text{Min}(B(R), B(S)) \leq M-1$ (or *M* as approximation)



What is the I/O Cost

$B(R) + B(S)$

Blocking vs. Non-Blocking Operators

- **Blocking operator** cannot produce any tuples to the output until it processes all its inputs
- **Non-blocking operator (Pipelined)** can produce tuples to output without waiting until all input is consumed

aka “on-the-fly”

- **Among operators we know, which ones are blocking ?**
 - Join, duplicate elimination, union → **Non-blocking**
 - Grouping → **Blocking**
 - Others? Selection, Projection → **Non-blocking**
 - Others? Sorting → **Blocking**

Summary

- Operator Evaluations:
 - Technique: Indexing, Iteration, Partitioning
 - #of passes: One Pass, Two Passes, Multi-Pass
 - Blocking vs. Non-blocking
- Examples:
 - Join(R, S)
 - Duplicate Elimination
 - Group by
 - Set Union(R, S)

CS4432: Database Systems II

Query Processing

Two-Pass Algorithms

Two-Pass Algorithms

First Pass: Do a prep-pass and write the intermediate result back to disk

>> **We count Reading + Writing**

Second Pass: Read from disk and compute the final results

>> **We count Reading only (if it is the final pass)**

- **Sort-based two-pass algorithms**
 - 1st pass does a sort on some parameter(s) of each operand
 - 2nd pass algorithm relies on sort results; can be pipelined
- **Hash-based two-pass algorithms**
 - 1st pass partitions input relation
 - 2nd pass algorithm will work on each partition independently

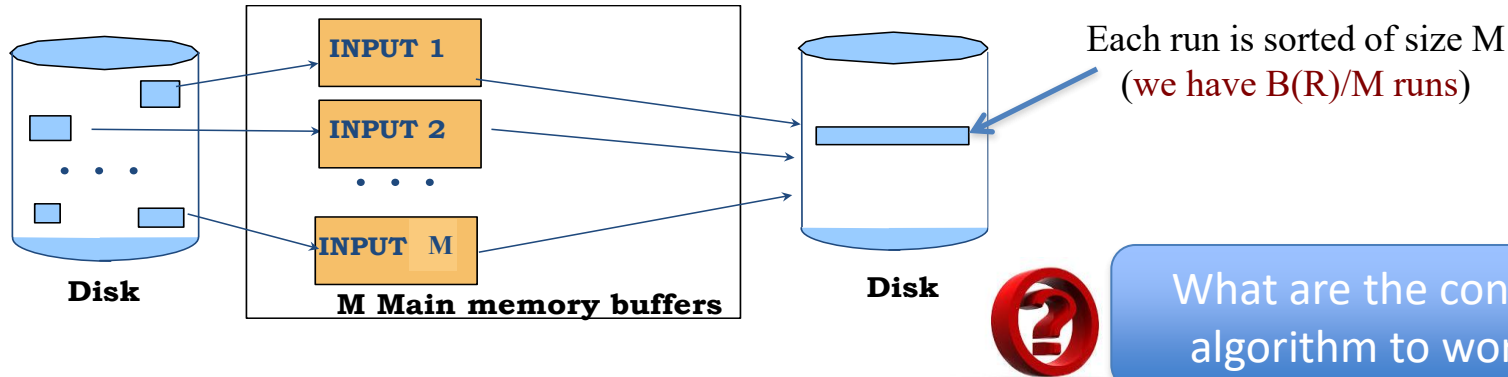
Sort

R

Example: 2-Pass External Sort

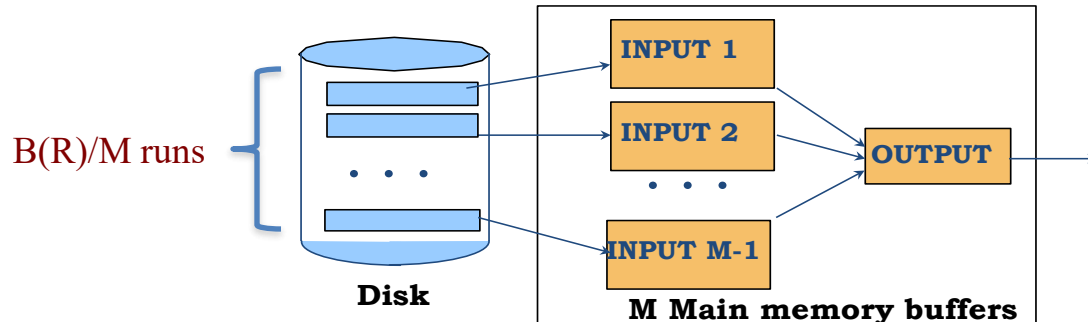
Reminder

Phase 1: Read M blocks at a time, sort them, write to disk as one run

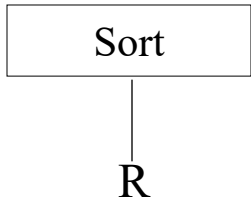


What are the constraints for this algorithm to work in 2 passes?

Phase 2: Merge the runs and produce the sorted output (each run must have one memory buffer)

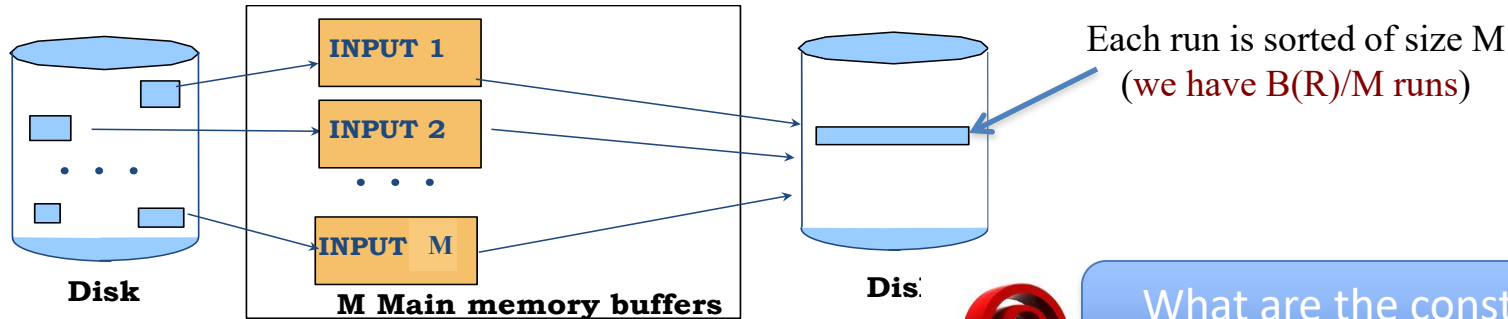


What is the I/O Cost



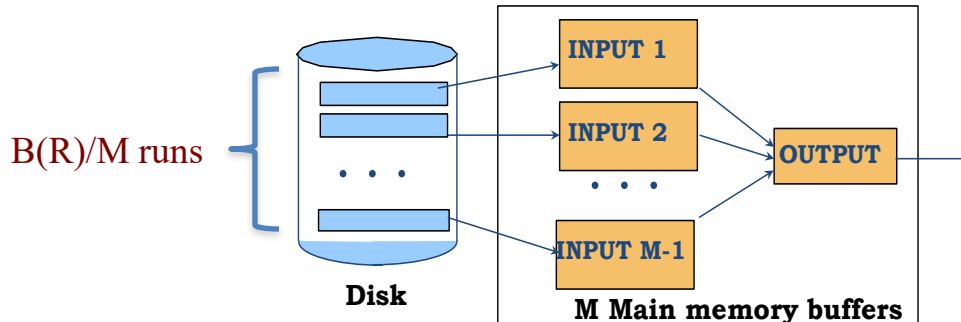
Example: 2-Pass External Sort

Phase 1: Read M blocks at a time, sort them, write to disk as one run



What are the constraints for this algorithm to work in 2 passes?

Phase 2: Merge the runs and produce the sorted output (each run must have one memory buffer)



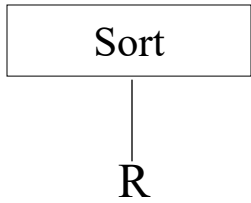
Phase 1 $\rightarrow$ no constraints

Phase 2 $\rightarrow$ each run must have a memory buffer + one for output

$$>> B(R)/M \leq M-1$$

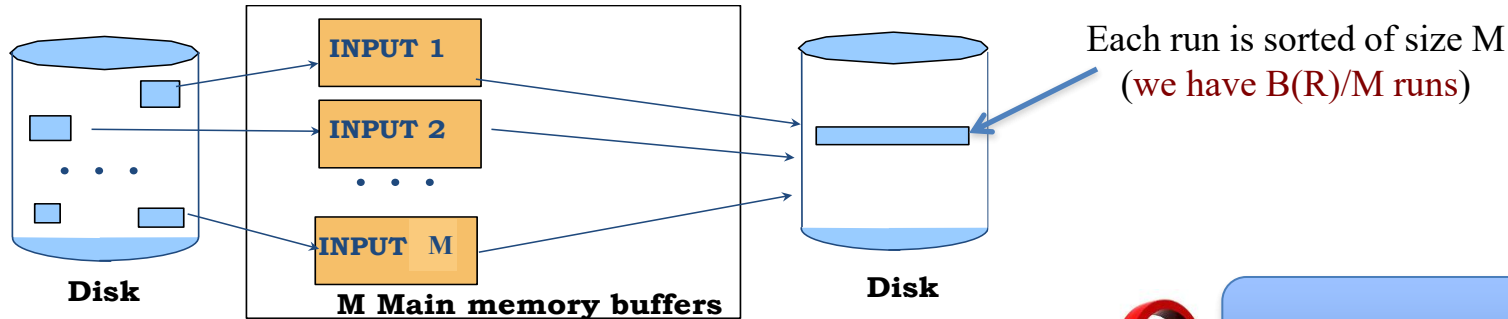
$$>> \text{Approx. } B(R)/M \leq M$$

$$>> B(R) \leq M^2$$



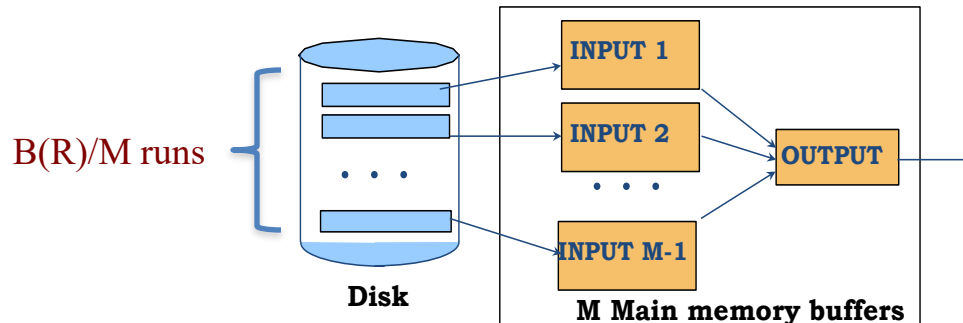
Example: 2-Pass External Sort

Phase 1: Read M blocks at a time, sort them, write to disk as one run



What is the I/O Cost

Phase 2: Merge the runs and produce the sorted output (each run must have one memory buffer)



Phase 1 $\rightarrow 2 \times B(R)$ [reading & writing]

Phase 2 $\rightarrow B(R)$ [reading]

Total $3 B(R)$

Distinct

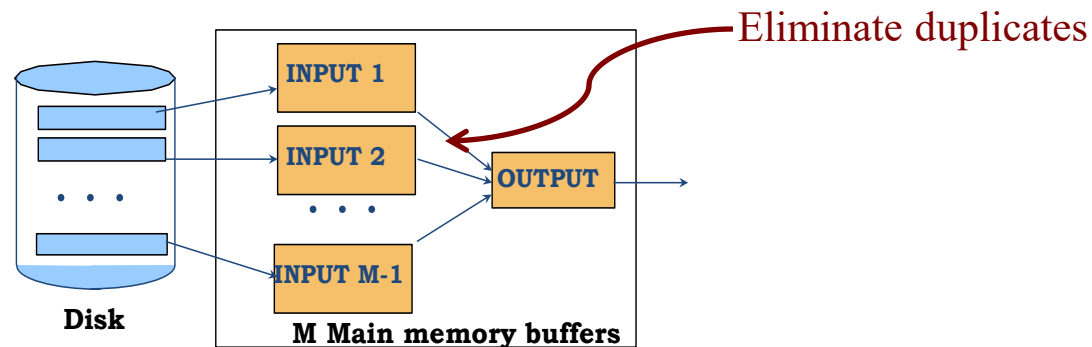
R

Iteration

Two Pass

Sort-Based Duplicate Elimination

- Same as sorting, except that:
 - While merging in Phase 2, eliminate the duplicates and produce one copy from each group of identical tuples

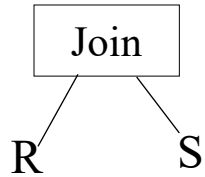


What are the constraints for this algorithm to work in 2 passes?



What is the I/O Cost

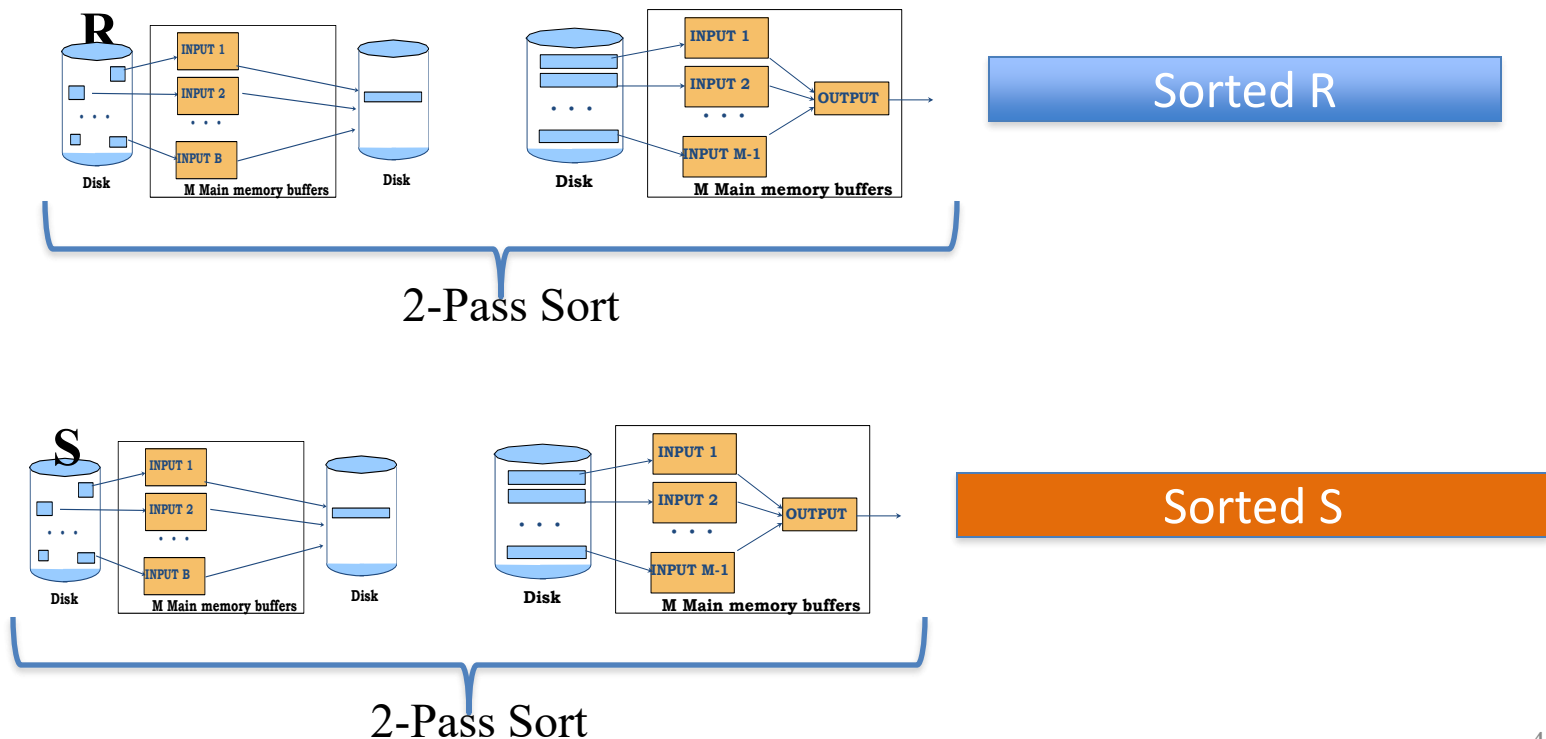
Same as the sorting operator itself

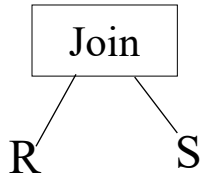


Naïve Two-Pass JOIN (*Sort-Join*)

1. Sort R and S on the join key (we assume equality join, e.g., $R.x = S.y$)
2. Merge and join the sorted R and S

Step 1 (Sorting each Relation)



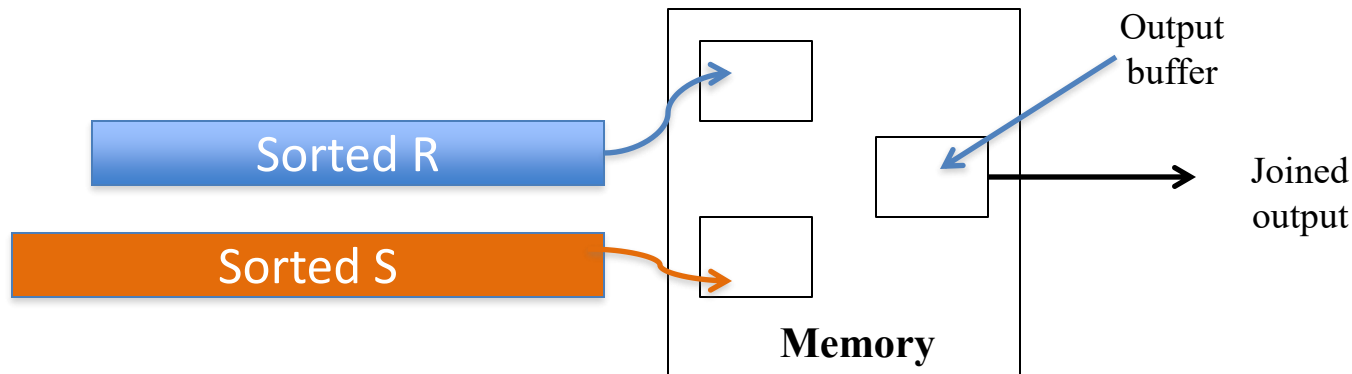


Naïve Two-Pass JOIN

1. Sort R and S on the join key
2. Merge and join the sorted R and S

Step 2 (Merge and Join R & S)

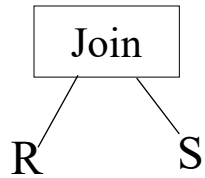
- Read one block from each relation at a time, join the tuples that exist in both relations
- When one block is consumed, read the next block from its relation



What are the constraints for this algorithm to work in two pass?



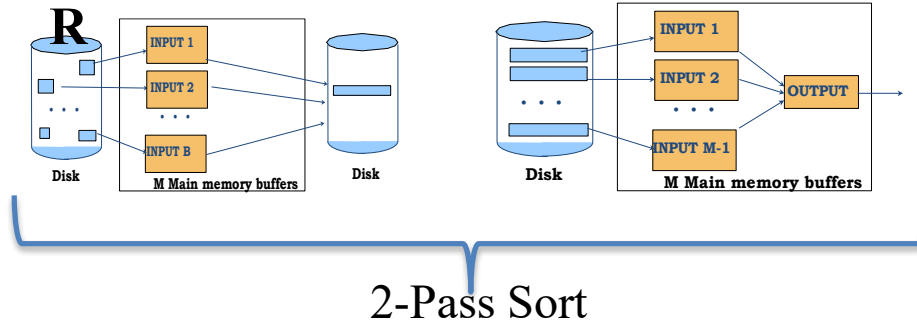
What is the I/O Cost



Naïve Two-Pass JOIN

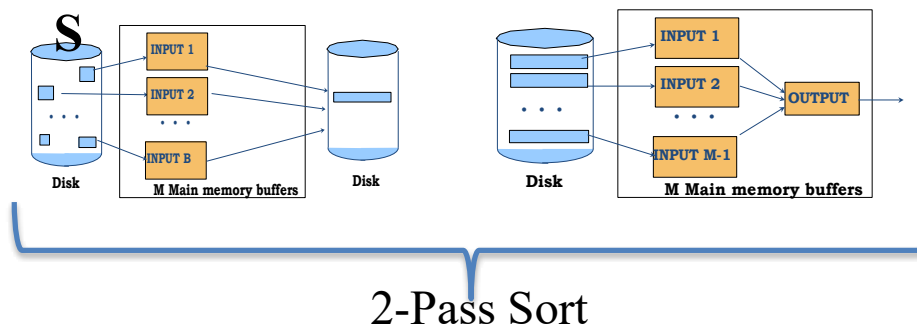


What is the I/O Cost

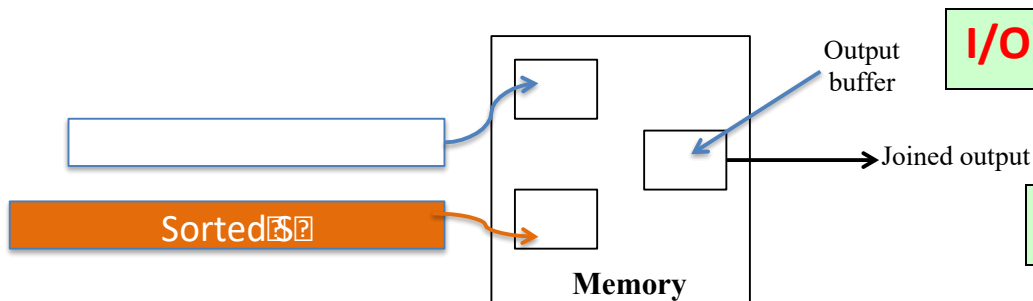


I/O Cost = 4 B(R)

Notice: we counted the output writing since it is intermediate

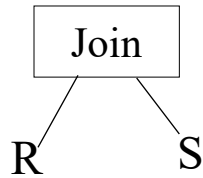


I/O Cost = 4 B(S)



I/O Cost = B(R) + B(S)

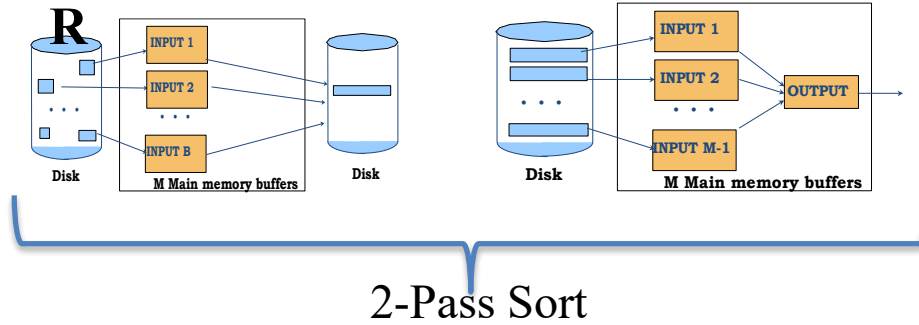
Total I/O Cost = 5(B(R) + B(S))



Naïve Two-Pass JOIN

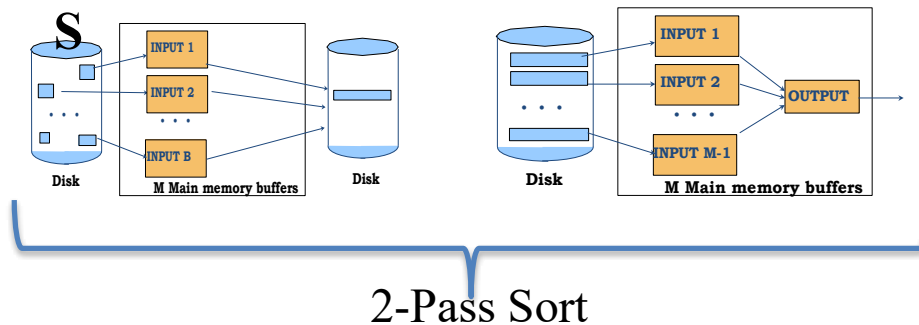


What are the constraints



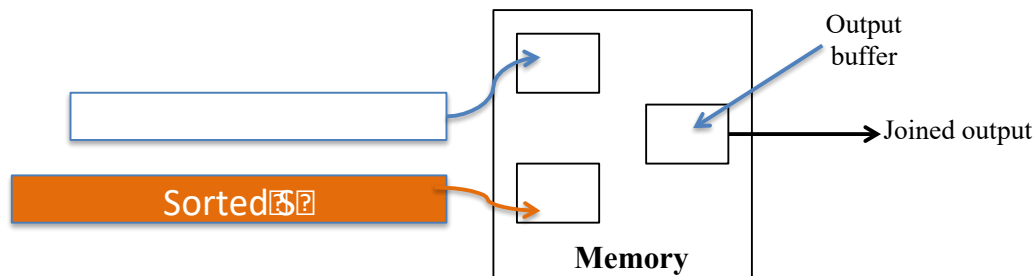
$$>> B(R) \leq M^2$$

From the sorting algorithm

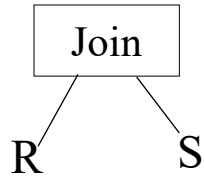


Sorted[S]

$$>> B(S) \leq M^2$$



No Constraints (at least 3 buffer pages available)



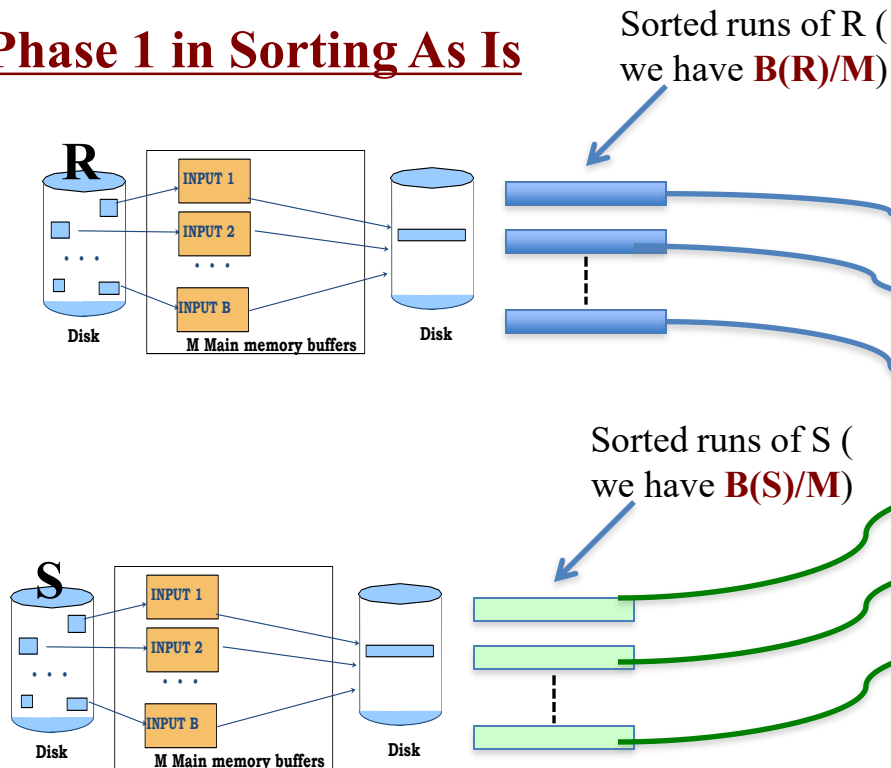
Efficient Two-Pass JOIN (*Sort-Merge-Join*)

Iteration

Two Pass

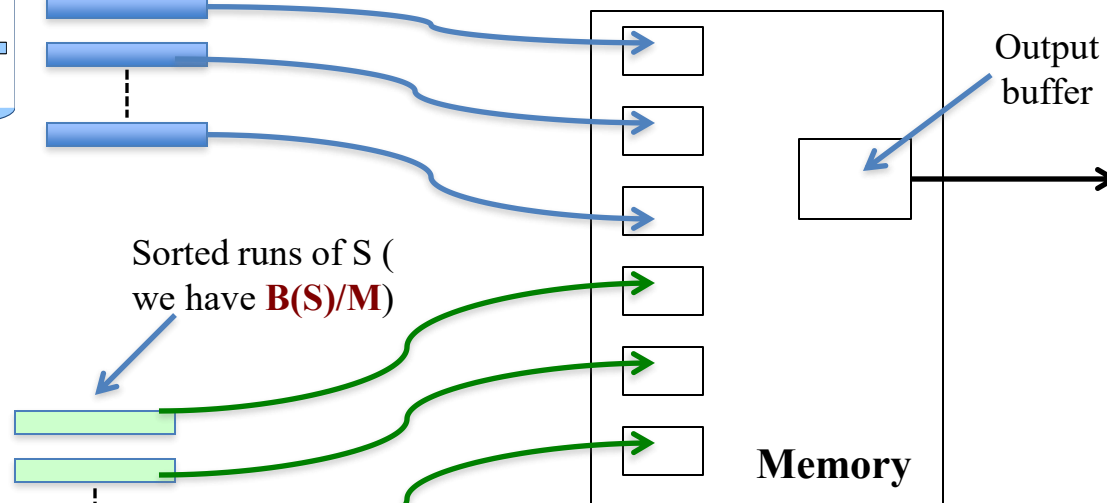
Main Idea: Combine Pass 2 of the Sort with the Join

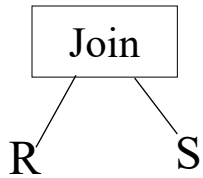
Phase 1 in Sorting As Is



Phase 2 Merge & Join

- One buffer for each sorted run from both R & S
- One buffer for the join output





Efficient Two-Pass JOIN

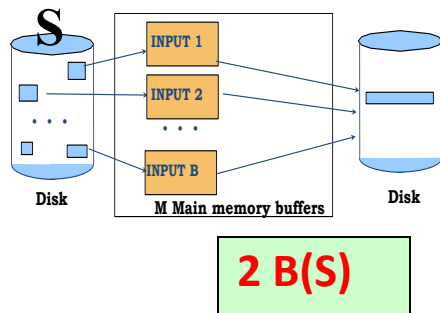
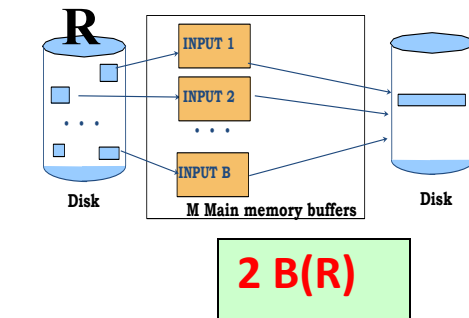
(*Sort-Merge-Join*)



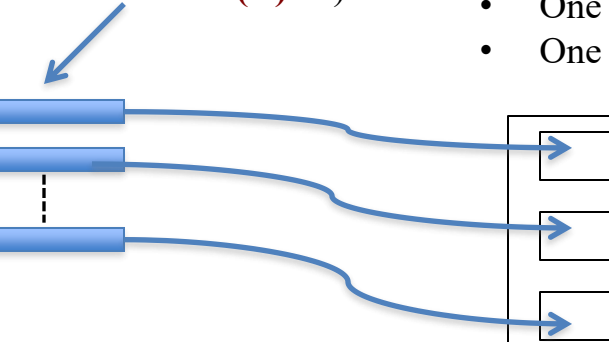
What is the I/O Cost

Main Idea: Combine Pass 2 of the Sort with the Join

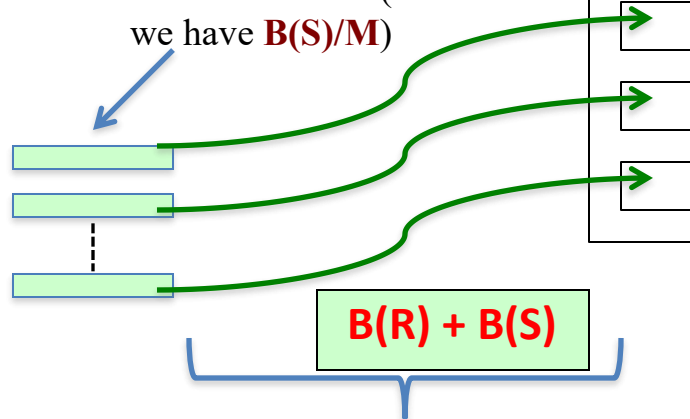
Phase 1 in Sorting As Is



Sorted runs of R (
we have $B(R)/M$)

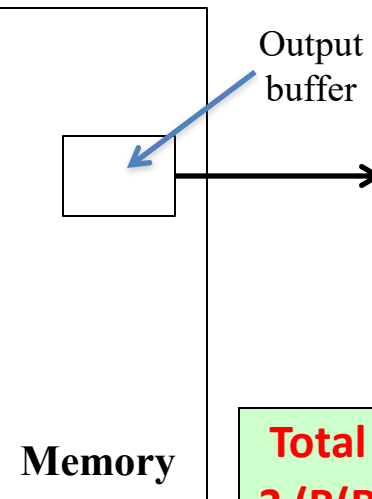


Sorted runs of S (
we have $B(S)/M$)

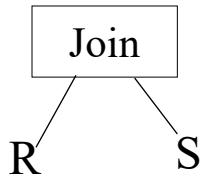


Phase 2 Merge & Join

- One buffer for each sorted run from both R & S
- One buffer for the join output



Total Cost =
 $3 (B(R) + B(S))$



Efficient Two-Pass JOIN

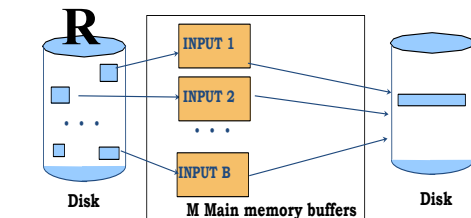
(*Sort-Merge-Join*)



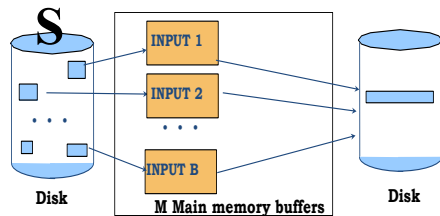
What are the constraints

Main Idea: Combine Pass 2 of the Sort with the Join

Phase 1 in Sorting As Is



No Constraints

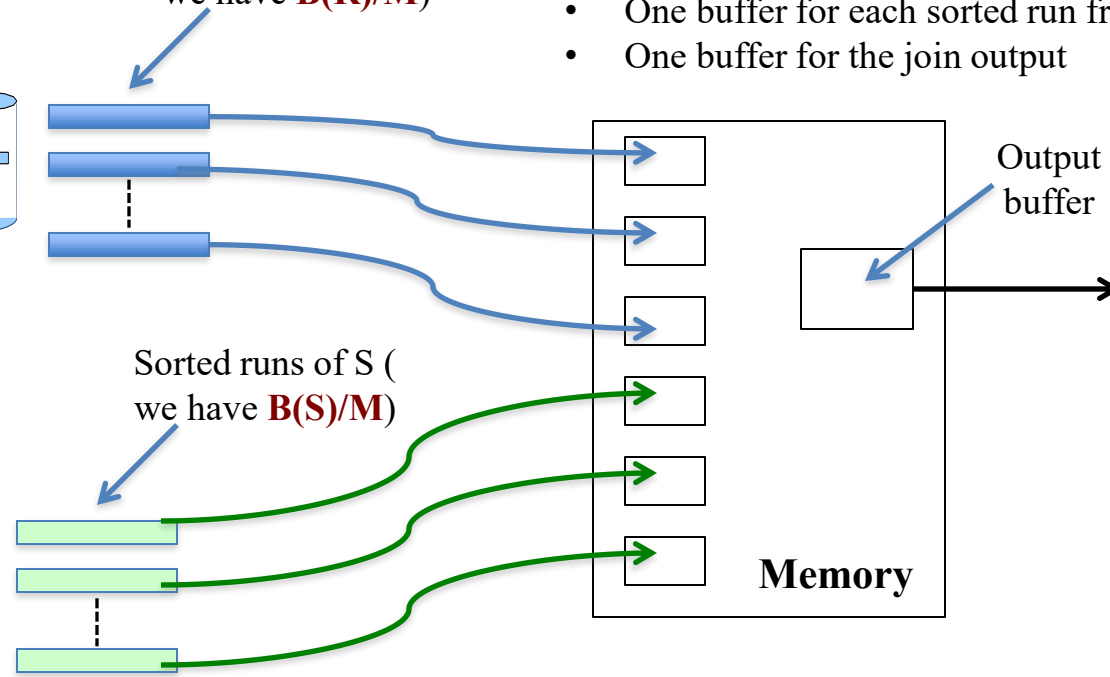


No Constraints

Sorted runs of R (we have $B(R)/M$)

Phase 2 Merge & Join

- One buffer for each sorted run from both R & S
- One buffer for the join output



Number of runs must fit in memory:

$$B(R)/M + B(S)/M \leq M \rightarrow B(R) + B(S) \leq M^2$$

Summary: Two-Pass Algorithms

First Pass: Do a prep-pass and write the intermediate result back to disk

>> **We count Reading + Writing**

Second Pass: Read from disk and compute the final results

>> **We count Reading only (if it is the final pass)**

- **Sort-based two-pass algorithms**

- 
- 1st pass does a sort on some parameter(s) of each operand
 - 2nd pass algorithm relies on sort results; can be pipelined

- **Hash-based two-pass algorithms**

- 1st pass partitions input relation
- 2nd pass algorithm will work on each partition independently

Compared to ONE PASS algorithms

- We pay more I/Os to execute the operator
- Can handle bigger relations